

Orange Digital Center

Rapport Application Python

DatasetManager

Baye Abdoul aziz seck
06/08/2026

Table des matières

Partie 1 : Types de base, variables, entrées et sorties	3
Les variables	3
Les types de données	3
Utilisation de Commentaires	4
Partie 2 : Structures de contrôle.....	6
La boucle while.....	6
Partie 3 : Dictionnaires	8
Partie 4 : Tuples.....	10
Partie 5 : Listes.....	13
4. Modifier un dataset	16
Partie 6 : Compréhensions (Listes et Dictionnaires).....	18
Partie 8 – Exceptions.....	22
Partie 9 : Fonctions	24
Paramètres avec valeur par défaut.....	25
Partie10 : Modules.....	26
Partie 11 : Packages	29
Partie 12 : Bonus	31

DatasetManager est une application console développée en Python dans le cadre de la formation **P1 Intelligence Artificielle**. L'objectif de cette application est de permettre la gestion d'un catalogue de jeux de données (datasets). Au fil des différentes parties du projet, de nouvelles fonctionnalités sont progressivement ajoutées afin de mettre en pratique les notions fondamentales de Python. Les principales fonctionnalités attendues sont :

- ajout d'un dataset ;
- affichage des datasets ;
- recherche d'un dataset ;
- modification des informations ;
- suppression d'un dataset ;
- affichage des statistiques ;
- sauvegarde dans un fichier ;
- rechargement automatique des données ;
- gestion des erreurs.

Technologies utilisées

- Python 3
- Terminal Windows
- Git
- GitHub
- PyCharm

Structure du projet

```
datasetManager/  
| — main.py
```

Partie 1 : Types de base, variables, entrées et sorties

Cette partie introduit les notions fondamentales de Python : variables, types de données (str, int, float, bool) et fonctions `input()` / `print()`. Consigne : Demander à l'utilisateur de saisir les métadonnées d'un dataset (nom, domaine, lignes, colonnes, taille, format, public) puis afficher un résumé formaté.

Cette première partie consiste à développer une application console permettant de saisir les principales métadonnées d'un dataset puis d'afficher un résumé des informations enregistrées.

Cette étape introduit les notions fondamentales de Python, notamment les variables, les types de données ainsi que les fonctions d'entrée et de sortie.

Les variables

Les variables permettent de stocker des informations en mémoire afin de pouvoir les réutiliser tout au long de l'exécution du programme.

En Python, la **déclaration** d'une variable et son **initialisation** (c'est-à-dire la première valeur que l'on va stocker dedans) se font en même temps.

Syntaxe

```
nom_variable = valeur
```

Exemple

```
nom_dataset = "Titanic"
```

Sans les variables, chaque information devrait être ressaisie à chaque utilisation.

Les types de données

Python offre plusieurs types de données de base que vous utiliserez fréquemment. Voici les principaux :

- **Entier (int)** : Utilisé pour stocker des nombres entiers, positifs ou négatifs.
- **Flottant (float)** : Utilisé pour stocker des nombres décimaux (avec une virgule flottante).
- **Chaîne de caractères (str)** : Utilisé pour stocker du texte, entouré de guillemets simples (') ou doubles (").
- **Booléen (bool)** : Utilisé pour représenter des valeurs de vérité (True ou False).

Types utilisés :

```
nom = "Abdou" # Une variable de type chaîne de caractères (str)
age = 30      # Une variable de type entier (int)
taille = 1.75 # Une variable de type flottant (float)
est_majeur = True # Une variable de type booléen (bool)
```

Chaque type possède un rôle précis dans le traitement des données.

```
entier = 42
flottant = 3.14
chaine = "Bonjour, Python !"
est_vrai = True
```

Conversion entre Types de Données

Vous pouvez convertir des variables d'un type à un autre en utilisant des fonctions de conversion. Par exemple :

```
nombre_texte = "123" # Une chaîne de caractères
nombre_entier = int(nombre_texte) # Convertit en entier
nombre_flottant = float(nombre_texte) # Convertit en flottant
# Vous pouvez également convertir des nombres en chaînes de caractères
age = 30
age_texte = str(age)
```

Assurez-vous de comprendre les types de données, car ils sont fondamentaux pour la manipulation des données en Python.

Utilisation de Commentaires

Les commentaires sont un moyen d'ajouter des explications dans votre code Python. Ils sont ignorés lors de l'exécution du programme. Utilisez le symbole # pour commencer un commentaire.

Exemple :

```
# Ceci est un commentaire
nom = "Alice" # Ceci est également un commentaire
```

NB :

« Python offre également la possibilité de rédiger des commentaires multilignes en encadrant le texte de triples guillemets. »

```
"""
Ceci est un exemple de texte long
"""
```

qui s'étend sur plusieurs lignes

et qui est ignoré par Python.

```
"""
```

La fonction input()

La fonction **input()** est une fonction intégrée qui permet de lire une ligne de texte depuis l'entrée standard et de la renvoyer en tant que chaîne de caractères. Elle est souvent utilisée pour demander à l'utilisateur d'entrer des données depuis la console.

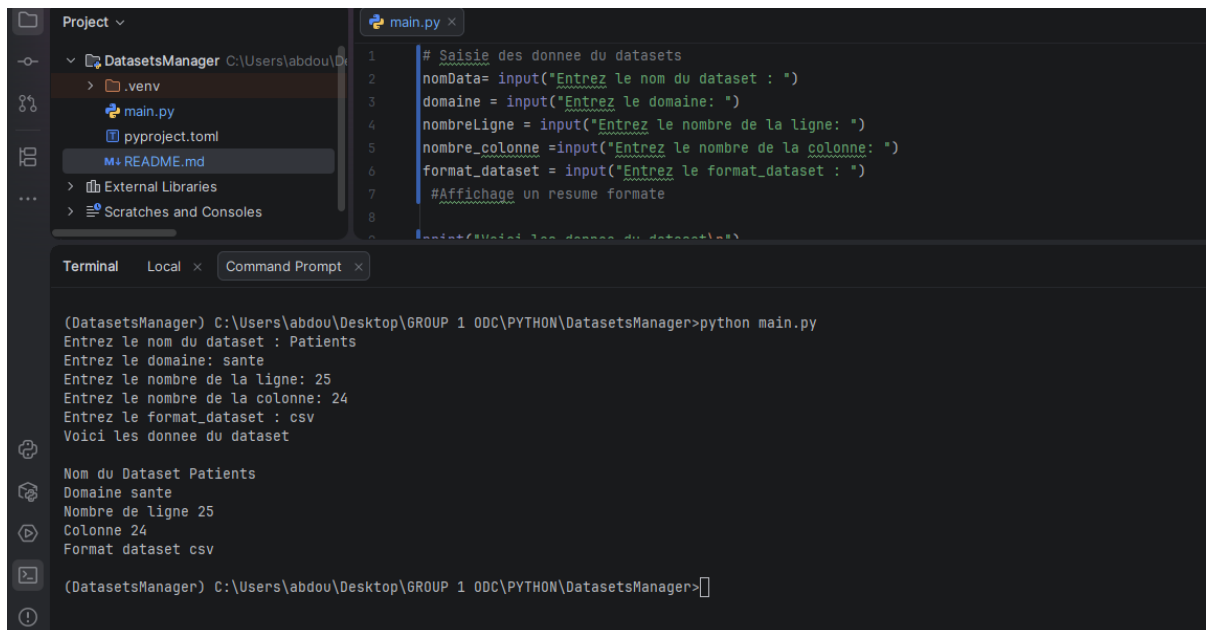
Syntaxe

```
input("Message")
```

Exemple

```
nom = input("Nom : ")
```

Sans cette fonction, aucune interaction avec l'utilisateur ne serait possible.



The screenshot shows a Python IDE with a file named `main.py` open. The code in the editor is as follows:

```
1 # Saisie des donnee du datasets
2 nomData= input("Entrez le nom du dataset : ")
3 domaine = input("Entrez le domaine: ")
4 nombreLigne = input("Entrez le nombre de la ligne: ")
5 nombre_colonne =input("Entrez le nombre de la colonne: ")
6 format_dataset = input("Entrez le format_dataset : ")
7 #Affichage un resume formate
8
9 print("Voici les donnee du dataset")
```

Below the editor, the terminal window shows the execution of the script. The prompt is `(DatasetsManager) C:\Users\abdou\Desktop\GROUP 1 ODC\PYTHON\DatasetsManager>python main.py`. The user enters the following values:

```
Entrez le nom du dataset : Patients
Entrez le domaine: sante
Entrez le nombre de la ligne: 25
Entrez le nombre de la colonne: 24
Entrez le format_dataset : csv
```

The script then prints the following summary:

```
Voici les donnee du dataset
Nom du Dataset Patients
Domaine sante
Nombre de ligne 25
Colonne 24
Format dataset csv
```

La fonction print()

Permet d'afficher des informations dans le terminal.

Syntaxe

```
print(valeur)
```

Exemple

```
print(nom)
```

Sans cette fonction, le programme effectuerait les traitements sans afficher les résultats.

Résultat obtenu

```
Terminal Local x Command Prompt x Command Prompt x
Entrez le format : csv
Entrez le public : oui

(DatasetsManager) PS C:\Users\abdou\Desktop\GROUP 1 ODC\PYTHON\DatasetsManager> python .\main.py
Entrez le nom du dataset : diabete
Entrez le domain du dataset : sante
Entrez le nombre de lignes : 15000
Entrez le nombre de colonnes : 25
Entrez le taille : 48
Entrez le format : csv
Entrez le public : non
{'nom': 'diabete', 'domaine': 'sante', 'lignes': 15000, 'colonnes': 25, 'taille': 48, 'format': 'csv', 'public': True}

(DatasetsManager) PS C:\Users\abdou\Desktop\GROUP 1 ODC\PYTHON\DatasetsManager> 
```

Cette première partie permet :

- la saisie des métadonnées d'un dataset ;
- le stockage des informations dans des variables ;
- l'affichage d'un résumé des données saisies.

Cette première partie introduit les bases de Python qui seront utilisées dans toutes les étapes suivantes du projet.

Partie 2 : Structures de contrôle

Cette partie transforme l'application en un programme interactif grâce à un menu permettant de sélectionner différentes actions.

Les notions abordées sont la boucle while, l'instruction match...case, le mot-clé break ainsi que le module os.

La boucle while

La boucle while permet de répéter un bloc d'instructions tant qu'une condition reste vraie.

Lorsque votre script rencontre une boucle while, il vérifie que la condition renvoie True.

Tant que cette condition retourne True, il exécute le code contenu à l'intérieur de la boucle sans interruption !

À chaque itération, il vérifie la condition et ne sort de la boucle que si elle renvoie False.

Syntaxe

while condition:

code

....

Exemple

i = 0

while i < 10:

print('Salut')

```
i += 1
```

Cette boucle maintient le programme actif jusqu'au choix de l'utilisateur de quitter l'application.

Sans cette boucle, le menu ne serait affiché qu'une seule fois.

L'instruction match...case

Permet d'exécuter un traitement différent selon la valeur saisie.

Syntaxe

match variable:

case valeur:

Chaque option du menu est associée à un case.

Cette structure améliore la lisibilité du programme.

Le mot-clé break

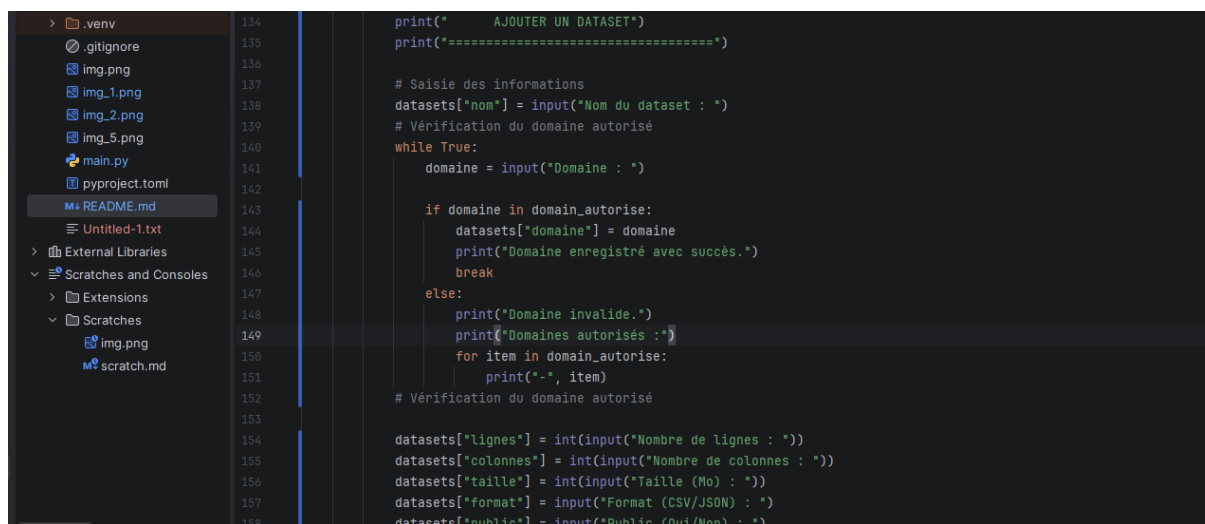
Permet d'interrompre immédiatement une boucle.

Syntaxe

break

Cette instruction est utilisée lorsque l'utilisateur choisit l'option **Quitter**.

Sans break, la boucle continuerait indéfiniment.



```
134 print("    AJOUTER UN DATASET")
135 print("=====")
136
137 # Saisie des informations
138 datasets["nom"] = input("Nom du dataset : ")
139 # Vérification du domaine autorisé
140 while True:
141     domaine = input("Domaine : ")
142
143     if domaine in domain_autorise:
144         datasets["domaine"] = domaine
145         print("Domaine enregistré avec succès.")
146         break
147     else:
148         print("Domaine invalide.")
149         print("Domaines autorisés : ")
150         for item in domain_autorise:
151             print("-", item)
152
153 # Vérification du domaine autorisé
154
155 datasets["lignes"] = int(input("Nombre de lignes : "))
156 datasets["colonnes"] = int(input("Nombre de colonnes : "))
157 datasets["taille"] = int(input("Taille (Mo) : "))
158 datasets["format"] = input("Format (CSV/JSON) : ")
159 datasets["public"] = input("Public (Oui/Non) : ")
```

Le module os

Le module os permet d'interagir avec le système d'exploitation.

Importation

import os

Effacer le terminal sous Windows

os.system("cls")

Cette instruction efface l'écran afin de proposer une interface plus propre et plus lisible.

Sous Linux ou macOS, la commande équivalente est :

```
os.system("clear")
```

Résultat obtenu

À l'issue de cette partie, l'application permet :

- d'afficher un menu interactif ;
- d'ajouter un dataset ;
- d'afficher les informations enregistrées ;
- de quitter proprement l'application.

Cette partie introduit les structures de contrôle nécessaires à la réalisation d'une application interactive en Python.

Partie 3 : Dictionnaires

Cette partie consiste à regrouper les différentes métadonnées d'un dataset dans une seule structure de données appelée dictionnaire.

Au lieu de manipuler plusieurs variables indépendantes, toutes les informations relatives à un même dataset sont désormais stockées dans un seul objet. Cette organisation rend le programme plus clair, plus facile à maintenir et prépare les prochaines étapes du projet.

Définition

Un dictionnaire est une collection **non ordonnée** (depuis Python 3.7, l'ordre d'insertion est préservé) de paires **clé-valeur**. Chaque clé est **unique** et pointe vers une valeur. Chaque information est identifiée par une clé unique, ce qui facilite son accès et sa modification.

Syntaxe

```
Nom_dictionnaire{} : dictionnaire vide
```

```
nom_dictionnaire = {  
    "clé": valeur  
}
```

La clé joue le rôle d'étiquette : elle doit être **unique** dans le dictionnaire (impossible d'avoir deux clés identiques) et **immuable** (les chaînes, nombres et tuples fonctionnent, mais pas les listes).

La valeur peut être de **n'importe quel type** Python : une chaîne, un nombre, une liste, un autre dictionnaire, ou même une fonction. Cette flexibilité fait des dictionnaires une structure de données extrêmement polyvalente.

```
dataset = {  
    "nom": "Titanic",  
    "domaine": "Transport",  
    "lignes": 891,  
    "colonnes": 12,  
    "taille": 48,  
    "format": "CSV",  
    "public": True  
}
```

Syntaxe avec dict()

Le constructeur **dict()** crée un dictionnaire à partir d'**arguments nommés**, d'une liste de **tuples**, ou de deux séquences combinées avec **zip()**. Pratique quand les données viennent déjà sous forme de paires.

#À partir de mots-clés

```
utilisateur = dict(nom="Alice", email="alice.durand@laposte.net", age=30)
```

À partir de tuples

```
utilisateur = dict([("nom", "Alice"), ("email", "alice.durand@laposte.net")])
```

À partir de deux listes avec zip()

```
cles = ["nom", "email", "age"]
```

```
valeurs = ["Alice", "alice.durand@laposte.net", 30]
```

```
utilisateur = dict(zip(cles, valeurs))
```

Accéder à une valeur

Pour récupérer une information, on utilise la clé.

```
print(dataset["nom"])
```

Modifier une valeur

Une valeur peut être modifiée à tout moment.

Avant :

```
dataset["taille"] = 48
```

Après modification :

```
dataset["taille"] = 60
```

Ajouter une nouvelle information

Il est possible d'ajouter une nouvelle clé dans le dictionnaire.

```
dataset["auteur"] = "Baye Abdoul Aziz Seck"
```

Le dictionnaire contient maintenant une nouvelle information.

```
Terminal Local x Command Prompt x Command Prompt x
Entrez le format : csv
Entrez le public : oui

(DatasetsManager) PS C:\Users\abdou\Desktop\GROUP 1 ODC\PYTHON\DatasetsManager> python .\main.py
Entrez le nom du dataset : diabete
Entrez le domain du dataset : sante
Entrez le nombre de lignes : 15000
Entrez le nombre de colonnes : 25
Entrez le taille : 48
Entrez le format : csv
Entrez le public : non
{'nom': 'diabete', 'domaine': 'sante', 'lignes': 15000, 'colonnes': 25, 'taille': 48, 'format': 'csv', 'public': True}

(DatasetsManager) PS C:\Users\abdou\Desktop\GROUP 1 ODC\PYTHON\DatasetsManager> 
```

```
=====
      AJOUTER UN DATASET
=====
Nom du dataset : titanic
Domaine : transport
Nombre de lignes : 848
Nombre de colonnes : 25
Taille (Mo) : 25
Format (CSV/JSON) : csv
Public (Oui/Non) : oui

Le dataset a été enregistré avec succès.

Appuyez sur Entrée pour revenir au menu... 
```

Partie 4 : Tuples

Un **tuple** en Python est une **collection ordonnée et immuable** de valeurs. Il ressemble à une **liste**, mais ses éléments **ne peuvent pas être modifiés** une fois le tuple créé. Cette **immuabilité** est sa caractéristique centrale : elle garantit que les données restent constantes et permet d'utiliser un tuple comme **clé de dictionnaire**, ce qu'une liste ne permet pas.

```
mon_tuple = ("Python",)
```

Dans le projet **DatasetManager**, un tuple est utilisé pour stocker la liste des domaines autorisés d'un dataset. Contrairement à une liste, un tuple est une structure de données **ordonnée et immuable**, ce qui signifie que son contenu ne peut pas être modifié après sa création.

Cette caractéristique est particulièrement utile lorsque les données ne doivent pas être changées pendant l'exécution du programme.

Par exemple, les domaines disponibles dans l'application sont fixés à l'avance et ne doivent pas être modifiés par l'utilisateur.

Il est également possible de créer un tuple contenant un seul élément :

```
mon_tuple = ("Python",)
```

La virgule est obligatoire lorsqu'un tuple ne contient qu'un seul élément.

Caractéristiques d'un tuple

Le tuple possède plusieurs caractéristiques importantes :

- les éléments sont ordonnés ;
- les éléments sont indexés à partir de 0 ;
- un tuple est immuable, il ne peut pas être modifié après sa création ;
- un tuple peut contenir différents types de données ;
- plusieurs valeurs identiques peuvent être présentes dans un même tuple.

Accéder à un élément

Chaque élément est accessible grâce à son indice.

Exemple :

```
print(domain_autorise[0])
```

Résultat :

Santé

Pour accéder au troisième élément :

```
print(domain_autorise[2])
```

Résultat :

Agriculture

Parcourir un tuple

Il est possible de parcourir un tuple avec une boucle for.

Exemple :

```
for domaine in domain_autorise:
```

```
    print(domaine)
```

Résultat :

Santé

Finance

Agriculture

Transport

Education

Tester la présence d'un élément

L'opérateur in permet de vérifier si une valeur appartient au tuple.

Exemple :

```
if "Transport" in domain_autorise:
```

```
print("Domaine valide")  
  
else:  
  
    print("Domaine invalide")
```

Pourquoi utiliser un tuple ?

Le tuple est utilisé lorsque les données sont fixes et ne doivent pas être modifiées.

Dans le projet **DatasetManager**, la liste des domaines autorisés reste toujours la même pendant l'exécution du programme.

L'utilisation d'un tuple permet donc de protéger ces valeurs contre toute modification accidentelle.

Application dans le TP DatasetManager

Dans notre application, un utilisateur ne peut enregistrer un dataset que si le domaine saisi appartient à la liste des domaines autorisés.

Le tuple est déclaré au début du programme :

```
domain_autorise = (  
    "Santé",  
    "Finance",  
    "Agriculture",  
    "Transport",  
    "Education"  
)
```

Lors de l'ajout d'un dataset, le programme demande le domaine puis vérifie qu'il existe dans le tuple.

```
while True:  
  
    domaine_saisi = input("Domaine : ").strip()  
  
    if domaine_saisi in domain_autorise:  
        datasets["domaine"] = domaine_saisi  
        break  
  
    print("Domaine invalide.")
```

Si le domaine saisi est présent dans le tuple, il est enregistré dans le dictionnaire du dataset.

Dans le cas contraire, le programme affiche un message d'erreur et redemande une nouvelle saisie.

Cette vérification permet de garantir que seuls les domaines prévus par l'application sont enregistrés.

```
domain_autorise = (  
    "Santé",  
    "Finance",  
    "Agriculture",  
    "Transport",  
    "Education",  
)
```

Partie 5 : Listes

Définition

Dans cette partie du projet **DatasetManager**, une liste est utilisée pour stocker plusieurs datasets. Chaque dataset est représenté par un dictionnaire contenant ses différentes informations (nom, domaine, nombre de lignes, nombre de colonnes, taille, format et visibilité).

L'utilisation d'une liste permet de gérer plusieurs datasets simultanément. À chaque ajout effectué par l'utilisateur, un nouveau dictionnaire est inséré dans cette liste. Toutes les opérations du programme (affichage, recherche, modification, suppression et tri) sont ensuite réalisées sur cette collection.

Définition d'une liste

Une liste est une structure de données ordonnée et modifiable qui permet de stocker plusieurs éléments dans une seule variable.

Les éléments sont indexés à partir de zéro et peuvent être de types différents (chaînes de caractères, nombres, dictionnaires, listes, etc.).

Syntaxe

Liste vide

```
list_datasets = []
```

Liste contenant des éléments

```
nombres = [10, 20, 30]
```

Liste contenant des dictionnaires

```
list_datasets = [  
    {  
        "nom": "Titanic",  
        "domaine": "Transport"  
    },  
    {  
        "nom": "Iris",  
        "domaine": "Agriculture"  
    }  
]
```

Application dans le TP DatasetManager

Dans notre application, chaque dataset est enregistré sous forme d'un dictionnaire.

Ces dictionnaires sont ensuite stockés dans la liste nommée :

```
list_datasets = []
```

Chaque nouvelle opération agit directement sur cette liste.

1. Ajouter un dataset

Objectif

Cette fonctionnalité permet d'enregistrer un nouveau dataset dans l'application.

Après la saisie des informations, le dictionnaire est ajouté dans la liste grâce à la méthode `append()`.

Code

```
datasets = {}  
  
datasets["nom"] = nom  
  
datasets["domaine"] = domaine  
  
datasets["lignes"] = lignes  
  
datasets["colonnes"] = colonnes  
  
datasets["taille"] = taille  
  
datasets["format"] = format_dataset  
  
datasets["public"] = public  
  
list_datasets.append(datasets)
```

```
8  
9      # Saisie du champ public.  
10     # Le sujet demande true ou false.  
11     while True:  
12         public_saisi = input("Public (true/false) : ").strip().lower()  
13  
14         if public_saisi in ("true", "vrai", "oui", "1"):  
15             datasets["public"] = True  
16             break  
17  
18         if public_saisi in ("false", "faux", "non", "0"):  
19             datasets["public"] = False  
20             break  
21  
22         print("Public invalide. Utilisez true ou false.")  
23  
24     # PARTIE 5 : LISTES  
25     # On ajoute le dictionnaire dans la liste des datasets.  
26     list_datasets.append(datasets)  
27
```

2. Afficher les datasets

Cette fonctionnalité permet d'afficher tous les datasets enregistrés.

Pour cela, une boucle parcourt la liste et affiche chaque dictionnaire.

```
for dataset in list_datasets:
```

```
    print(dataset["nom"])
```

```
    print(dataset["domaine"])
```

```
    print(dataset["lignes"])
```

```
case 2:
    print("=====")
    print("  INFORMATIONS DES DATASETS")
    print("=====")

    if not list_datasets:
        print("Aucun dataset enregistré pour le moment.")
    else:
        for dataset in list_datasets:
            print(f"Nom du dataset      : {dataset['nom']}")
            print(f"Domaine              : {dataset['domaine']}")
            print(f"Nombre de lignes       : {dataset['lignes']}")
            print(f"Nombre de colonnes     : {dataset['colonnes']}")
            print(f"Taille (Mo)            : {dataset['taille']}")
            print(f"Format                 : {dataset['format']}")
            print(f"Public                 : {dataset['public']}")
            print(separateur)

        input("\nAppuyez sur Entrée pour revenir au menu...")

# =====
```

3. Rechercher un dataset

La recherche permet de retrouver un dataset à partir de son nom.

Le programme parcourt la liste et compare le nom saisi avec celui de chaque dataset.

Code

```
recherche = input("Nom : ")
```

```
for dataset in list_datasets:
```

```
    if recherche.lower() == dataset["nom"].lower():
```

```
        print("Dataset trouvé")
```

```
        break
```



```

322         if not list_datasets:
323             print("Aucun dataset enregistré pour le moment.")
324         else:
325             recherche = input("Entrez le nom du dataset : ").strip()
326             trouve = False
327
328             for dataset in list_datasets:
329                 if recherche.lower() == dataset["nom"].lower():
330                     print("\nDataset trouvé.\n")
331                     print(f"Nom du dataset      : {dataset['nom']}")
332                     print(f"Domaine          : {dataset['domaine']}")
333                     print(f"Nombre de lignes   : {dataset['lignes']}")
334                     print(f"Nombre de colonnes : {dataset['colonnes']}")
335                     print(f"Taille (Mo)        : {dataset['taille']}")
336                     print(f"Format            : {dataset['format']}")
337                     print(f"Public            : {dataset['public']}")
338                     print(separateur)
339                     trouve = True
340                     break
341             if not trouve:
342                 print("\nDataset introuvable.")
343

```

4. Modifier un dataset

Cette fonctionnalité permet de modifier les informations d'un dataset existant.

Une fois le dataset trouvé, ses différentes valeurs sont remplacées par les nouvelles saisies de l'utilisateur.

```

    if not list_datasets:
        print("Aucun dataset enregistré pour le moment.")
    else:
        nom_a_modifier = input("Entrez le nom du dataset à modifier : ").strip()
        trouve = False

        for dataset in list_datasets:
            if nom_a_modifier.lower() == dataset["nom"].lower():
                trouve = True

                print(f"\nDataset '{dataset['nom']}' trouvé.")
                print("Laissez vide si vous ne souhaitez pas modifier une valeur.\n")

                # Modification du nom.
                nouveau_nom = input(f"Nouveau nom ({dataset['nom']}) : ").strip()

                if nouveau_nom != "":
                    if ";" in nouveau_nom:
                        print("Le nom ne doit pas contenir de ';'. Nom non modifié.")
                    else:
                        dataset["nom"] = nouveau_nom

                # Modification du domaine.

```

for dataset in list_datasets:

if dataset["nom"].lower() == nom_a_modifier.lower():

dataset["nom"] = nouveau_nom

dataset["domaine"] = nouveau_domaine

5. Supprimer un dataset

Cette fonctionnalité permet de retirer définitivement un dataset de la liste.

Le programme utilise la méthode `pop()` après confirmation de l'utilisateur.

Code

```
print("-----")
print("      SUPPRIMER UN DATASET")
print("=====")
if not list_datasets:
    print("Aucun dataset enregistré pour le moment.")
else:
    nom_a_supprimer = input("Entrez le nom du dataset à supprimer : ").strip()
    trouve = False

    for index, dataset in enumerate(list_datasets):
        if nom_a_supprimer.lower() == dataset["nom"].lower():
            trouve = True
            print("\nDataset trouvé :)")
            print(f"Nom du dataset      : {dataset['nom']}")
            print(f"Domaine                : {dataset['domaine']}")
            print(f"Nombre de lignes       : {dataset['lignes']}")
            print(f"Nombre de colonnes     : {dataset['colonnes']}")
            print(f"Taille (Mo)            : {dataset['taille']}")
            print(f"Format                 : {dataset['format']}")
            print(f"Public                 : {dataset['public']}")
            print(separateur)

            confirmation = input("Confirmer la suppression ? (Oui/Non) : ").strip().lower()

            if confirmation in ("oui", "true", "1", "vrai"):
                list_datasets.pop(index)
                print("Dataset supprimé avec succès.")
```

6. Trier les datasets

Objectif

Le tri permet d'organiser les datasets par ordre alphabétique selon leur nom.

Dans notre projet, un tri manuel (tri à bulles) a été utilisé afin de mettre en pratique les structures de contrôle étudiées.

Code

```
n = len(list_datasets)
for i in range(n):
    for j in range(n - i - 1):
        if list_datasets[j]["nom"].lower() > list_datasets[j + 1]["nom"].lower():
            temp = list_datasets[j]
            list_datasets[j] = list_datasets[j + 1]
            list_datasets[j + 1] = temp
```

L'utilisation des listes constitue une étape importante dans le développement de l'application **DatasetManager**. Contrairement aux parties précédentes où un seul dataset pouvait être manipulé, la liste permet désormais de gérer un ensemble de datasets. Grâce à cette structure, il est possible d'ajouter, d'afficher, de rechercher, de modifier, de supprimer et de trier les données de manière organisée. Cette évolution rend l'application plus réaliste, plus flexible et prépare les prochaines étapes du projet, notamment la sauvegarde et le chargement des données à partir d'un fichier CSV.

```
526     if not list_datasets:
527         print("Aucun dataset enregistré pour le moment.")
528     else:
529         # Tri manuel sans fonction personnalisée ni lambda.
530         # On utilise une méthode simple de tri à bulles.
531         n = len(list_datasets)
532
533         for i in range(n):
534             for j in range(0, n - i - 1):
535                 nom1 = list_datasets[j]["nom"].lower()
536                 nom2 = list_datasets[j + 1]["nom"].lower()
537
538                 if nom1 > nom2:
539                     list_datasets[j], list_datasets[j + 1] = list_datasets[j + 1], list_datasets[j]
540
541         print("Datasets triés par nom avec succès.\n")
542
543         for dataset in list_datasets:
544             print(f"Nom du dataset      : {dataset['nom']}")
545             print(f"Domaine          : {dataset['domaine']}")
546             print(f"Nombre de lignes   : {dataset['lignes']}")
547             print(f"Nombre de colonnes : {dataset['colonnes']}")
548             print(f"Taille (Mo)        : {dataset['taille']}")
549             print(f"Format             : {dataset['format']}")
550             print(f"Public             : {dataset['public']}")
551             print(separateur)
```

Partie 6 : Compréhensions (Listes et Dictionnaires)

Une compréhension de liste (List Comprehension) est une manière simple et concise de créer une nouvelle liste à partir d'une liste existante.

Elle permet de remplacer plusieurs lignes de code utilisant une boucle for par une seule instruction plus lisible.

Les compréhensions de listes sont souvent utilisées pour parcourir une collection, filtrer des données ou effectuer des transformations.

Syntaxe générale

nouvelle_liste = [expression for element in liste]

Avec une condition :

nouvelle_liste = [expression for element in liste
if condition]

Exemple simple

Créer une liste contenant les carrés des nombres de 1 à 5.

```
nombres = [1, 2, 3, 4, 5]
carres = [nombre * nombre for nombre in nombres]
print(carres)

Résultat :

[1, 4, 9, 16, 25]
```

Exemple avec une condition

Créer une liste contenant uniquement les nombres pairs.

```
nombres = [1,2,3,4,5,6,7,8]
pairs = [nombre for nombre in nombres
         if nombre % 2 == 0]
print(pairs)

Résultat :

[2,4,6,8]
```

Application dans le projet DatasetManager

Dans notre projet, les compréhensions de listes peuvent être utilisées pour extraire certaines informations de tous les datasets.

Par exemple, récupérer uniquement les noms des datasets enregistrés.

```
noms = [dataset["nom"] for dataset in list_datasets]
print(noms)

Résultat :
```

```
['Titanic', 'Iris', 'Wine']
```

On peut également récupérer uniquement les datasets publics.

```
datasets_publics = [
    dataset
    for dataset in list_datasets
    if dataset["public"] == True
]
```

Ou encore récupérer uniquement les tailles.

```
tailles = [dataset["taille"] for dataset in list_datasets]

Résultat :

[48, 120, 35]
```

Avantages des compréhensions de listes

Les compréhensions de listes présentent plusieurs avantages :

- elles réduisent le nombre de lignes de code ;
- elles rendent le programme plus lisible ;
- elles facilitent la création de nouvelles listes ;
- elles permettent de filtrer des données rapidement ;
- elles améliorent la productivité du développeur.

Dans **DatasetManager**, cette notion peut être utilisée pour :

- récupérer tous les noms des datasets ;
- obtenir uniquement les datasets publics ;
- récupérer les tailles de tous les datasets ;
- extraire les formats utilisés (CSV ou JSON) ;
- préparer des statistiques sur les données.

```
45
46 list_datasets = []
47
```

```
if not list_datasets:
    print("Aucun dataset enregistré pour le moment.")
else:
    nombre_datasets = len(list_datasets)

    # Compréhension de liste : lignes de chaque dataset.
    liste_lignes = [dataset["lignes"] for dataset in list_datasets]
    total_lignes = sum(liste_lignes)

    # Compréhension de liste : colonnes de chaque dataset.
    liste_colonnes = [dataset["colonnes"] for dataset in list_datasets]
    moyenne_colonnes = sum(liste_colonnes) / nombre_datasets

    # Compréhension de liste : datasets publics / privés.
    datasets_publics = [dataset for dataset in list_datasets if dataset["public"] is True]
    datasets_prives = [dataset for dataset in list_datasets if dataset["public"] is False]

    # Compréhension de liste : datasets par format.
    datasets_csv = [dataset for dataset in list_datasets if dataset["format"] == "CSV"]
    datasets_json = [dataset for dataset in list_datasets if dataset["format"] == "JSON"]

    # Compréhension de dictionnaire : répartition par domaine.
```

```
datasets_publics = [
    dataset
    for dataset in list_datasets
    if dataset["public"] == True
]
```

```

571 # Compréhension de liste : datasets publics / privés.
572 datasets_publics = [dataset for dataset in list_datasets if dataset["public"] is True]
573 datasets_prives = [dataset for dataset in list_datasets if dataset["public"] is False]
574
575 # Compréhension de liste : datasets par format.
576 datasets_csv = [dataset for dataset in list_datasets if dataset["format"] == "CSV"]
577 datasets_json = [dataset for dataset in list_datasets if dataset["format"] == "JSON"]
578
579 # Compréhension de dictionnaire : répartition par domaine.
580 repartition_domaines = {}
581     domaine: len([dataset for dataset in list_datasets if dataset["domaine"] == domaine])
582     for domaine in domain_autorise
583 }
584
585 print(f"Nombre de datasets      : {nombre_datasets}")
586 print(f"Nombre total de lignes   : {total_lignes}")
587 print(f"Nombre moyen de colonnes    : {moyenne_colonnes:.0f}")
588 print(f"Datasets publics           : {len(datasets_publics)}")
589 print(f"Datasets privés            : {len(datasets_prives)}")
590 print(f"Datasets au format CSV     : {len(datasets_csv)}")
591 print(f"Datasets au format JSON    : {len(datasets_json)}")
592 print("\nRépartition par domaine :")
593
594     for domaine, total in repartition_domaines.items():
595         print(f"    {domaine} : {total}")
596
597 input("\nAppuyez sur Entrée pour revenir au menu...")

```

Partie 7 :Fichiers

Cette partie consiste à sauvegarder les datasets dans un fichier CSV afin de conserver les données après la fermeture du programme. Au démarrage, le programme vérifie également si un fichier de sauvegarde existe et recharge automatiquement les datasets enregistrés.

Le format **CSV** (Comma Separated Values) est un format texte utilisé pour stocker des données sous forme de tableau. Dans ce projet, le caractère ; est utilisé comme séparateur entre les différentes colonnes.

Le fichier contient une première ligne appelée **en-tête**, suivie des données des différents datasets.

Exemple de contenu du fichier :

nom	domaine	lignes	colonnes	taille	format	public
1 diabet	Santé	1587	45	1	CSV	True

Les principales fonctionnalités réalisées sont :

- Chargement automatique du fichier au démarrage du programme.
- Sauvegarde de tous les datasets dans un fichier datasets.csv.
- Lecture du fichier ligne par ligne.
- Découpage des informations grâce à la méthode split(";").
- Conversion des données numériques avec int().
- Reconstruction des dictionnaires puis ajout dans la liste des datasets.

```

if os.path.exists(fichier_sauvegarde):

with open(fichier_sauvegarde, "r", encoding="utf-8") as fichier:

lignes_fichier = fichier.readlines()

valeurs = ligne.split(";")

dataset = {}

dataset["nom"] = valeurs[0]

dataset["domaine"] = valeurs[1]

with open(fichier_sauvegarde, "w", encoding="utf-8") as fichier:

fichier.write(

    f'{dataset["nom"]};{dataset["domaine"]};{dataset["lignes"]};'

    f'{dataset["colonnes"]};{dataset["taille"]};'

    f'{dataset["format"]};{dataset["public"]}\n'

)

```

Résultat obtenu

À la fin de cette partie, le programme est capable de sauvegarder automatiquement les datasets dans un fichier CSV et de les recharger lors de son exécution. Les données ne sont donc plus perdues après la fermeture du programme, ce qui rend l'application plus pratique et plus proche d'un véritable outil de gestion de données.

```

if not list_datasets:
    print("Aucun dataset à sauvegarder.")
else:
    try:
        with open(fichier_sauvegarde, "w", encoding="utf-8") as fichier:
            fichier.write(f"{nom};domaine;lignes;colonnes;taille;format;public\n")

            for dataset in list_datasets:
                ligne = (
                    f'{dataset["nom"]};{dataset["domaine"]};{dataset["lignes"]};'
                    f'{dataset["colonnes"]};{dataset["taille"]};{dataset["format"]};'
                    f'{dataset["public"]}\n'
                )
                fichier.write(ligne)

            print("Sauvegarde effectuée avec succès dans datasets.csv.")

    except PermissionError:
        print("Erreur : impossible d'écrire dans le fichier (accès refusé).")

    except OSError:
        print("Erreur : une erreur est survenue lors de l'écriture du fichier.")

input("\nAppuyez sur Entrée pour revenir au menu...")

```

Partie 8 – Exceptions

Les exceptions permettent de gérer les erreurs qui peuvent survenir pendant l'exécution d'un programme. Au lieu d'arrêter brutalement le programme lorsqu'une erreur se produit, Python permet de la détecter et d'exécuter un traitement approprié.

Dans le projet **Dataset Manager**, les exceptions sont utilisées pour éviter que le programme ne se ferme lorsqu'un utilisateur saisit une valeur incorrecte.

Syntaxe

```
try:  
    ... # Code susceptible de provoquer une exception  
except NomDeLException:  
    ... # Code à exécuter en cas d'exception
```

Utilisation dans le projet

Lors de la saisie du menu principal, l'utilisateur doit entrer un nombre. Si une lettre ou un autre caractère est saisi, une exception est déclenchée.

Exemple :

```
try:  
    choix = int(input("Entrez votre choix : "))  
except ValueError:  
    print("Choix invalide. Veuillez entrer un nombre.")
```

Dans cet exemple :

- **try** contient le code pouvant provoquer une erreur.
- **int()** tente de convertir la saisie en entier.
- Si la conversion échoue, Python lève une exception **ValueError**.
- Le bloc **except** affiche un message d'erreur au lieu d'interrompre le programme.

Avantages

- Empêche l'arrêt brutal du programme.
- Améliore la robustesse de l'application.
- Guide l'utilisateur lorsqu'une erreur de saisie est détectée.
- Rend le programme plus fiable et plus agréable à utiliser.

Résultat obtenu

Grâce à la gestion des exceptions, **Dataset Manager** continue de fonctionner même lorsqu'un utilisateur effectue une saisie incorrecte. Le programme affiche un message explicatif et permet à l'utilisateur de recommencer sans redémarrer l'application.

```
Choix invalide. Veuillez entrer un nombre entre 1 et 10.  
Appuyez sur Entrée pour revenir au menu...█
```


Partie 9 : Fonctions

Pensez à une fonction comme à une **machine** : vous lui donnez quelque chose en **entrée**, elle fait son travail, et vous récupérez un **résultat en sortie**. Parfois, elle peut juste faire une **action** sans rien vous rendre.

En Python, une fonction ressemble à ceci :

Syntaxe :

```
def nom_de_la_fonction():  
    # Ce que fait la fonction  
    print("Hello !")
```

Le mot **def** dit à Python : "Je vais créer une **nouvelle fonction**". Après, vous choisissez un **nom** (ici **nom_de_la_fonction**), puis vous mettez des **parenthèses ()**.

Exemple :

```
def dire_bonjour():  
    print("Bonjour ! Comment allez-vous ?")
```

Pour utiliser cette fonction (on dit **l'appeler**), écrivez simplement :

nom_fonction()

Fonctions avec paramètres

Une fonction peut recevoir des informations appelées paramètres. Ces paramètres permettent à la fonction de travailler avec différentes valeurs.

```
def afficher_dataset(dataset : {__getitem__}): new *  
    print(dataset["nom"])  
    print(dataset["domaine"])
```

Appeler la fonction

```
print(dataset["domaine"])  
# Appeler la fonction  
afficher_dataset(dataset)
```

Paramètres avec valeur par défaut

Une fonction peut également posséder une valeur par défaut. Si aucun argument n'est fourni lors de son appel, cette valeur sera utilisée automatiquement.

Exemple :

```
1 def saluer(nom : str = "Utilisateur"): 2 usages new *
2     print("Bonjour", nom)
3 # Utilisation :
4 saluer("Baye")
5 saluer()
6
```

Le premier appel affiche **Bonjour Baye**, tandis que le second affiche **Bonjour Utilisateur**.

Retour d'une valeur avec return

Certaines fonctions calculent une valeur puis la renvoient grâce au mot-clé return.

Exemple :

```
1 def addition(a : {__add__}, b): 1 usage new *
2     return a + b
3 Utilisation :
4 resultat = addition(a: 5, b: 3)
5 print(resultat)
6
```

Le résultat affiché est 8.

Dans le projet **Dataset Manager**, plusieurs fonctions utilisent return pour renvoyer un résultat qui sera utilisé par le programme.

```
91 #FONCTION POUR AJOUTER DATASET
92 def ajouter_dataset(): 1 usage new *
93     print("=====")
94     print("    AJOUTER UN DATASET")
95     print("=====")
96
97     # PARTIE 3 : DICTIONNAIRE
98     # Un dataset est stocké sous forme de dictionnaire.
99     datasets = {}
100
101     # Saisie du nom du dataset.
102     while True:
103         nom = input("Nom du dataset : ").strip()
104
105         if nom == "":
106             print("Le nom du dataset ne peut pas être vide.")
107
108         elif ";" in nom:
109             print("Le nom ne doit pas contenir de ';' car il sera sauvegardé en CSV.")
110
111         else:
112             break
113
114     datasets[nom] = nom
115
116     # Saisie du domaine avec vérification par rapport au tuple.
```

```

640 # =====
641 # CASE 1 : AJOUTER UN DATASET
642 # =====
643 case 1:
644     ajouter_dataset()
645     input("\nAppuyez sur Entrée pour revenir au menu...")
646 # =====
647 # CASE 2 : AFFICHER LES DATASETS
648 # =====
649 case 2:
650     print("=====")
651     print("  INFORMATIONS DES DATASETS")
652     print("=====")
653     Afficher_dataset()
654     input("\nAppuyez sur Entrée pour revenir au menu...")
655 # =====
656 # CASE 3 : RECHERCHER UN DATASET
657 # =====
658 case 3:
659     rechercher_dataset()
660     input("\nAppuyez sur Entrée pour revenir au menu...")
661 # =====
662 # CASE 4 : MODIFIER UN DATASET
663 # =====
664 case 4:
665

```

Partie10 : Modules

Un module Python est un fichier ayant l'extension **.py** qui contient du code Python, comme des variables, des fonctions ou des classes. Les modules permettent de séparer un programme en plusieurs fichiers afin de mieux organiser le code et de faciliter sa maintenance.

Dans le projet **Dataset Manager**, les modules sont utilisés pour répartir les différentes fonctionnalités de l'application dans plusieurs fichiers au lieu de tout écrire dans un seul fichier `main.py`.

Pourquoi utiliser des modules ?

L'utilisation des modules présente plusieurs avantages :

- Organiser le projet en plusieurs fichiers.
- Éviter d'avoir un fichier principal trop volumineux.
- Réutiliser les mêmes fonctions dans plusieurs programmes.
- Faciliter la maintenance et les évolutions de l'application.

Création d'un module

Un module est simplement un fichier Python.

Par exemple, on peut créer un fichier **gestion_datasets.py** contenant les fonctions de gestion des datasets.

gestion_datasets.py

```

def afficher_dataset(dataset):
    print(dataset["nom"])
    print(dataset["domaine"])

```

Ce fichier devient automatiquement un module Python.

Importer un module

Une fois le module créé, il peut être utilisé dans un autre fichier grâce au mot-clé **import**.

Dans le projet **Dataset Manager**, le fichier principal **main.py** importe les fonctions présentes dans les autres modules.

Exemple :

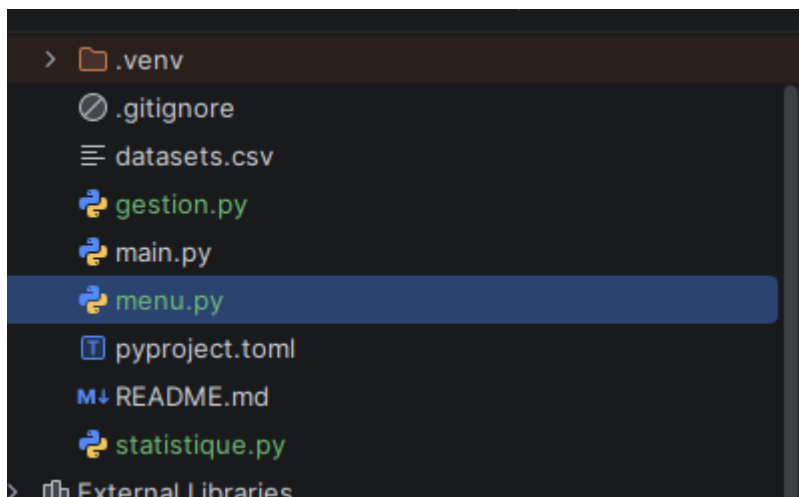
```
import gestion_datasets  
  
gestion_datasets.afficher_dataset(dataset)
```

Il est également possible d'importer uniquement une fonction.

```
from gestion_datasets import afficher_dataset  
  
afficher_dataset(dataset)
```

Organisation du projet

Après l'utilisation des modules, le projet est organisé en plusieurs fichiers.



Chaque fichier possède un rôle précis :

main.py

Le fichier main.py est le point d'entrée du programme.

Son rôle est de :

- lancer l'application ;
- afficher le menu principal ;
- appeler les fonctions présentes dans les autres modules.

Il contient très peu de logique métier.

menu.py

Le fichier menu.py contient tout ce qui concerne l'affichage du menu.

Par exemple :

- affichage des différentes options ;

récupération du choix de l'utilisateur.

Cela évite de mélanger le menu avec le reste du programme.

gestion.py

Le fichier gestion.py contient les principales fonctionnalités du Dataset Manager.

Toutes les opérations réalisées sur les datasets sont regroupées dans ce module.

- **statistique.py**

Le fichier statistique.py contient uniquement les traitements statistiques.

Séparer ces calculs dans un module rend le programme plus lisible.

Importation des modules

Pour utiliser les fonctions contenues dans un autre fichier, Python utilise le mot-clé import.

Exemple :

import gestion

import menu

import statistique

Ou encore :

```
from gestion import ajouter_dataset
```

Cette instruction permet d'utiliser directement la fonction sans écrire le nom du module.

Avantages obtenus

L'utilisation des modules apporte plusieurs avantages :

code plus propre ;

meilleure organisation du projet ;

maintenance plus simple ;

réutilisation des fonctions ;

évolution plus facile du programme ;

travail collaboratif facilité.

La Partie 10 a permis de transformer un programme constitué d'un seul fichier en une application organisée en plusieurs modules spécialisés.

Chaque module possède un rôle précis :

main.py pilote l'application ;

menu.py gère le menu ;

gestion.py gère les opérations sur les datasets ;

statistique.py réalise les calculs statistiques.

Cette organisation respecte les bonnes pratiques de développement Python et facilite les évolutions futures du projet.

Partie 11 : Packages

Un package est un dossier contenant plusieurs modules Python liés entre eux. Il permet d'organiser un projet en regroupant les fichiers selon leurs fonctionnalités.

Contrairement à un module qui correspond à un seul fichier Python (.py), un package est constitué de plusieurs modules placés dans un même répertoire.

Dans le projet **Dataset Manager**, le découpage du programme en plusieurs fichiers permet de mieux organiser le code et de faciliter sa maintenance.

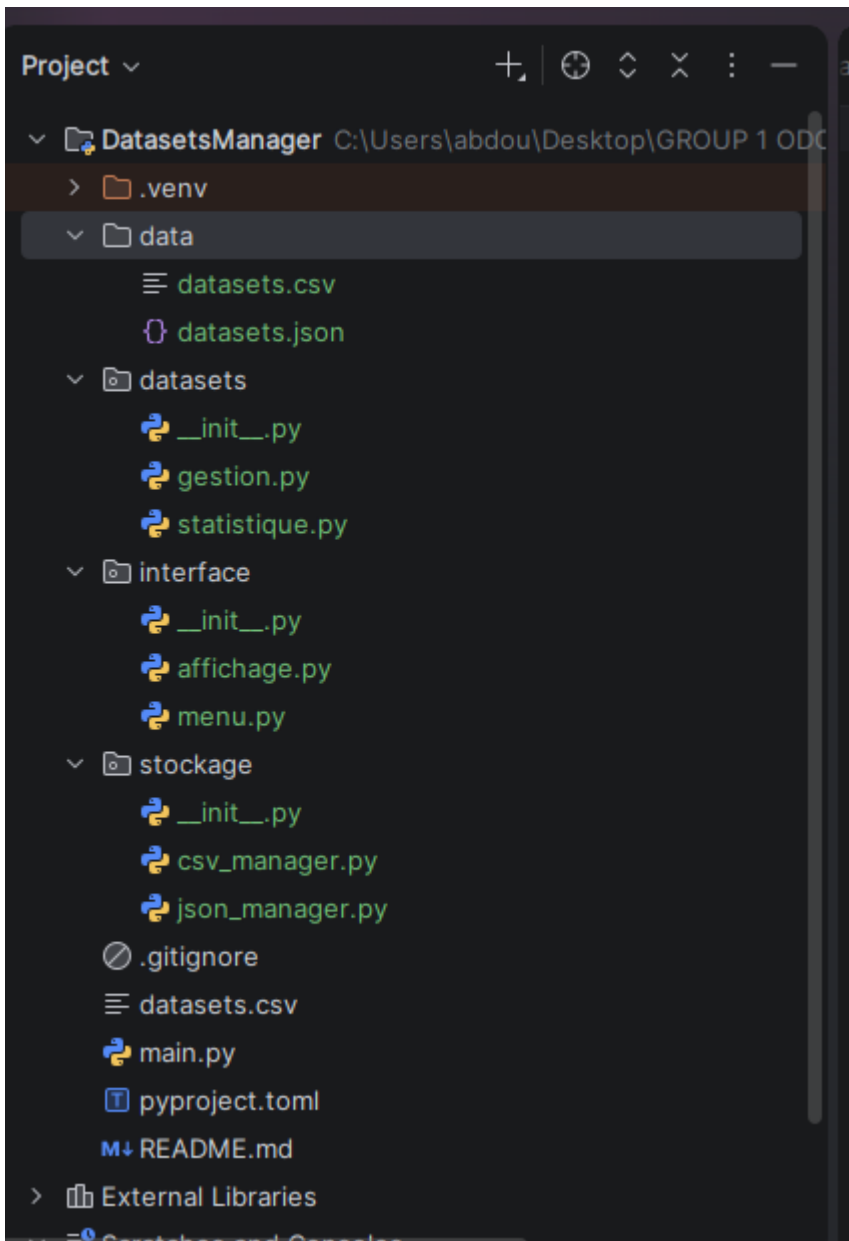
Pourquoi utiliser un package ?

Les packages permettent de :

- organiser un projet en plusieurs modules ;
- rendre le code plus lisible ;
- faciliter la réutilisation des fonctionnalités ;
- simplifier la maintenance du programme ;
- permettre le travail collaboratif sur différents fichiers.

Structure utilisée dans le projet

Le projet Dataset Manager est organisé en plusieurs fichiers :



Chaque fichier possède un rôle précis :

- **main.py** : point d'entrée du programme.
- **menu.py** : contient les fonctions d'affichage du menu.
- **gestion.py** : gère les opérations sur les datasets (ajout, modification, suppression, recherche).
- **statistique.py** : calcule les statistiques des datasets.
- **datasets.csv** : stocke les données.
- **pyproject.toml** : décrit le projet Python.

Importation des modules

Les modules sont importés dans le fichier principal avec l'instruction `import`.

Exemple :

```
import menu
```

```
import gestion
```

```
import statistique
```

On peut ensuite utiliser les fonctions définies dans ces modules.

Exemple :

```
menu.afficher_menu()
```

```
gestion.ajouter_dataset()
```

```
statistique.afficher_statistiques()
```

Avantages des packages

L'utilisation d'un package présente plusieurs avantages :

- le code est mieux organisé ;
- chaque module possède une responsabilité précise ;
- les modifications sont plus simples à réaliser ;
- plusieurs développeurs peuvent travailler sur différents modules en même temps ;
- le projet devient plus évolutif.

Conclusion

La Partie 11 a permis d'organiser le projet **Dataset Manager** en plusieurs modules regroupés dans une même structure. Cette organisation améliore la lisibilité du code, facilite sa maintenance et prépare le projet à évoluer vers des applications Python plus importantes.

Partie 12 : Bonus